1 Memory Addresses

1.1 Consider the following C program:

```
int a = 5;
int main()
{
    int b = 0;
    char* s1 = "cs61c";
    char s2[] = "cs61c";
    char* c = malloc(sizeof(char) * 100);
    return 0;
};
```

For each of the following values, state the location in the memory layout where they are stored. Answer with code, static, heap, or stack.

(a) `s1`

(b) `s2`

(c) `s1[0]`

(d) `s2[0]`

(e) `c[0]`

(f) `a`

1.2 Consider the C code here, and assume the malloc call succeeds. Rank the value of each variable (which may be an address!) from 1 to 5 (with 1 being the least) right before bar returns. Use the memory layout from class; Treat all addresses as unsigned numbers.

```
#include <stdlib.h>

int FIVE = 5;

int bar(int x) {
    return x * x;
}

int main(int argc, char *argv[]) {
    int *foo = malloc(sizeof(int));
    if (foo) free(foo);
    bar(10); //  snapshot just before it returns
    return 0;
}
```

```
foo:   _______
&foo:  _______
FIVE:  _______
&FIVE: _______
&x:    _______
```

2 Linked Lists Revisited

2.1 Fill out the declaration of a singly linked linked-list node below.

```
typedef struct node {
    int value;
    ____________ next; // pointer to the next element
} sll_node;
```

2.2 Let's convert the linked list to an array. Fill in the missing code.

```
int* to_array(sll_node *sll, int size) {
    int i = 0;
    int *arr = _________________;
    while (sll) {
        arr[i] = ___________;
        sll = ___________;
        _______;
    }
    return arr;
}
```

2.3 Finally, complete the function `delete_even()` that will delete every second element of the list. For example, given the lists below:

```
Before: Node 1 → Node 2 → Node 3 → Node 4
After: Node 1 → Node 3
```

Calling `delete_even()` on the list labeled "Before" will change it into the list labeled "After". All list nodes were created via dynamic memory allocation.

```
void delete_even(sll_node *s11) {
    sll_node *temp;
    if (!sll || !sll->next) {
        return;
    }
    temp = ________;
    sll->next = ________;
    free(________);
    delete_even(________);
}
```

3 Floating Point Intro

The IEEE standard defines a binary representation for floating point using **sign**, **exponent**, and **mantissa**.

Sign	Exponent	Significand
1 bit	8 bits	23 bits

For normalized floats:
Value = $(-1)^{Sign} \times 2^{(Exponent - Bias)} \times 1.significand_2$

For denormalized floats:
Value = $(-1)^{Sign} \times 2^{(Exponent - Bias + 1)} \times 0.significand_2$

Exponent	**Significand**	**Meaning**
0	Anything	Denorm
1-254	Anything	Normal
255	0	Infinity
255	Nonzero	NaN

3.1 (a) How would 10.625 be represented in floating point format?

(b) What decimal number is encoded as `0xC0A80000`?

(c) How many non-negative floats are strictly less than 2?

(d) What is the smallest positive value that can be stored using a single precision float?

4 More Floating Point (Sp18 Final)

4.1 IEEE 754-2008 introduces half precision, which is a binary floating-point representation that uses 16 bits: 1 sign bit, 5 exponent bits (with a bias of 15) and 10 significand bits. This format uses the same rules for special numbers that IEEE754 uses. Considering this half-precision floating point format, answer the following questions:

(a) For 16-bit half-precision floating point, how many different valid representations are there for NaN?

(b) What is the smallest positive non-infinite number it can represent? You can leave your answer as an expression.

(c) What is the largest non-infinite number it can represent? You can leave your answer as an expression.

(d) How many floating point numbers are in the interval [1, 2) (including 1 but excluding 2?

5 RISC-V Basics

5.1 Write the equivalent C statements in RISC-V. Assume the following:

- The variable 'x' resides in register 'x1'
- The variable 'y' resides in register 'x2'
- The variable 'z' resides in register 'x3'
- You are free to use the register 'x4' to hold temporary values

(a) `z = x + y`

(b) `x = x + 1`

(c) `y = y - 1`

(d) `x = x + 0`

(e) `x = y - z + x - 4`